Diabetes Risk Predictor

End-to-End MLOps Pipeline

From Raw Data to Live AWS Deployment

Data Preprocessing · Model Training · Hyperparameter Tuning · FastAPI · Docker · AWS EC2 · AWS S3 · AWS ECR

Author	Sharan

Dataset	Pima Indians Diabetes Database (UCI ML Repository)
Best Model	Logistic Regression
AUC Score	0.8399
Accuracy	76.4%
Deployed	AWS EC2 (us-east-1) via Docker + FastAPI
Status	**Live**

Built as part of an AI/ML learning journey. Not intended for clinical use.

Contents

1 Project Goal and Motivation **3**

2 Dataset Overview **3**
2.1 Source . . . 3
2.2 Characteristics . . . 3
2.3 Features . . . 4
2.4 Class Imbalance . . . 4

3 Complete Pipeline Architecture **4**

4 Data Preprocessing **5**
4.1 Missing Value Handling — MICE Imputation . . . 5
4.2 Class Imbalance — SMOTE . . . 5

5 Model Training and Evaluation **6**
5.1 Cross-Validation Strategy . . . 6
5.2 Models Compared . . . 6
5.3 Metric Priorities . . . 6
5.4 MLflow Experiment Tracking . . . 6

6 Hyperparameter Tuning **6**
6.1 Methods Used . . . 7
6.2 Tuning Results . . . 7
6.3 Key Observation . . . 7

7 API Development — FastAPI **7**
7.1 Architecture . . . 7
7.2 Request / Response Structure . . . 8
7.3 Middleware . . . 8
7.4 Model Loading Strategy . . . 8

8 Cloud Deployment **8**
8.1 Infrastructure Overview . . . 8
8.2 Deployment Steps . . . 8
8.3 Docker Strategy . . . 9

9 Frontend Interface **9**

10 Latency Analysis **9**
10.1 Measurement Methodology . . . 9
10.2 Measured Results . . . 9
10.3 Key Finding . . . 10
10.4 Optimisations Applied . . . 10
10.5 Projected Improvement . . . 10

11 Tools and Technologies **11**

12 What Was Learnt **11**

13 Results Summary **12**

14 Conclusion **12**

1 Project Goal and Motivation

The goal of this project was never just to train a model. It was to understand what it actually takes to build and deploy a machine learning system from scratch — handling messy data, comparing multiple algorithms, tracking experiments properly, and shipping a real API that anyone can call from a browser.

Diabetes affects over 500 million people worldwide. Early detection through accessible screening tools can meaningfully reduce long-term complications. The Pima Indians Diabetes Database from the UCI Machine Learning Repository provided a compact but realistic clinical dataset to work with — eight features, 768 records, and a meaningful class imbalance problem.

The project was structured around a simple question: *can a complete stranger open a browser, enter their health details, and get an AI-powered risk assessment in under a second?* That question drove every technical decision from preprocessing to deployment.

2 Dataset Overview

2.1 Source

The Pima Indians Diabetes Database is publicly available on Kaggle and the UCI Machine Learning Repository. It was originally collected by the National Institute of Diabetes and Digestive and Kidney Diseases.

2.2 Characteristics

Property	Value
Total records	768 patients
Features	8 clinical measurements
Target variable	Outcome (0 = No Diabetes, 1 = Diabetes)
Positive cases	268 (34.9%)
Negative cases	500 (65.1%)
Missing values	Present (encoded as zeros)

Table 1: Dataset summary statistics

2.3 Features

Feature	Description	Unit
Pregnancies	Number of times pregnant	Count
Glucose	Plasma glucose concentration (2h OGTT)	mg/dL
BloodPressure	Diastolic blood pressure	mm Hg
SkinThickness	Triceps skin fold thickness	mm
Insulin	2-hour serum insulin	μU/mL
BMI	Body mass index	kg/m^2
DiabetesPedigreeFunction	Genetic risk score	0.08 – 2.42
Age	Age of patient	Years

Table 2: Feature descriptions

2.4 Class Imbalance

The dataset has a 65:35 split between non-diabetic and diabetic cases. This imbalance means a naive model could achieve 65% accuracy by simply predicting no diabetes every time. This made SMOTE oversampling a necessary component of the training pipeline.

3 Complete Pipeline Architecture

The full pipeline spans six distinct stages, from raw CSV ingestion to a live AWS-hosted prediction API.

```
Raw CSV Data (768 records)
        ↓
[Stage 1] Exploratory Data Analysis
  - Missing value identification
  - Distribution analysis
  - Class imbalance check
        ↓
[Stage 2] Data Preprocessing
  - MICE Imputation (IterativeImputer)
  - Save imputed dataset to CSV
        ↓
[Stage 3] Model Training & Evaluation
  - Stratified 5-Fold Cross Validation
  - SMOTE applied per fold (training only)
  - 5 algorithms compared
  - MLflow experiment tracking
        ↓
[Stage 4] Hyperparameter Tuning
  - GridSearchCV
  - RandomizedSearchCV
  - Optuna (50 trials per model)
  - Best model saved as .pkl
```

```
        ↓
[Stage 5] API Development
  - FastAPI backend (/predict endpoint)
  - Pydantic input validation
  - Model loaded from AWS S3
  - CORS + GZip middleware
        ↓
[Stage 6] Cloud Deployment
  - Dockerized application
  - Image pushed to AWS ECR
  - Running on AWS EC2
  - Model stored in AWS S3
  - Frontend served locally
        ↓
User Browser → Form → API → Prediction
```

4 Data Preprocessing

4.1 Missing Value Handling — MICE Imputation

Several features in the dataset use zero as a placeholder for missing values — physiologically impossible readings such as a glucose level of 0 or a BMI of 0. These were identified and treated as missing.

Standard approaches like mean or median imputation ignore relationships between features. MICE (Multiple Imputation by Chained Equations), implemented via scikit-learn's `IterativeImputer`, treats each feature with missing values as a regression target and predicts it using the remaining features. This preserves inter-feature correlations.

Listing 1: MICE Imputation

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

imputer = IterativeImputer(random_state=42, max_iter=10)
X_imputed = imputer.fit_transform(X_raw)
```

The imputed dataset was saved to CSV so that downstream training scripts could load clean data directly without repeating the imputation step.

4.2 Class Imbalance — SMOTE

With 268 positive and 500 negative cases, SMOTE (Synthetic Minority Oversampling Technique) was applied. SMOTE generates synthetic minority class samples by interpolating between existing ones, rather than simply duplicating them.

Critical implementation detail: SMOTE was applied *inside* each cross-validation fold, only to training data. Applying it before the fold split would constitute data leakage — the model would have seen synthetic versions of test-set neighbours during training.

5 Model Training and Evaluation

5.1 Cross-Validation Strategy

Stratified 5-Fold Cross Validation was used throughout. Stratification ensures that each fold maintains the original 65:35 class ratio, preventing any fold from being accidentally dominated by one class.

5.2 Models Compared

Five algorithms were trained and evaluated:

Model	AUC	Accuracy	F1	Precision	Recall
Logistic Regression	**0.8398**	0.7642	**0.6902**	0.6497	**0.7424**
Random Forest	0.8277	0.7564	0.6705	0.6378	0.7089
CatBoost	0.8257	0.7682	0.6799	0.6574	0.7051
Gradient Boosting	0.8238	0.7539	0.6607	0.6371	0.6865
XGBoost	0.8119	0.7512	0.6549	0.6365	0.6751

Table 3: Model comparison — 5-fold cross-validation results

Logistic Regression outperformed all ensemble methods. This is a common outcome on small, linearly separable datasets — complex models need more data to justify their additional parameters.

5.3 Metric Priorities

In a medical screening context, Recall is the most critical metric. A false negative — predicting no diabetes in a patient who has it — is considerably more dangerous than a false positive. The selected model achieves 74.2% recall, meaning it correctly identifies nearly three in four actual diabetic cases.

5.4 MLflow Experiment Tracking

Every training run was logged to MLflow, including:

- Per-fold and mean metrics (AUC, Accuracy, F1, Precision, Recall)
- Hyperparameters used
- ROC curve plots (saved as artifacts)
- SHAP feature importance plots
- Final trained model (logged as MLflow model artifact)

6 Hyperparameter Tuning

Three tuning methods were applied to Logistic Regression and Random Forest:

6.1 Methods Used

1. **GridSearchCV** — exhaustive search over a predefined parameter grid. Reliable but slow for large grids.
2. **RandomizedSearchCV** — samples random combinations from the parameter space. Faster than grid search with competitive results.
3. **Optuna** — Bayesian optimisation framework. Learns from previous trials to intelligently navigate the search space. 50 trials per model.

6.2 Tuning Results

Model	Method	AUC	Accuracy	F1	Precision	Recall
Logistic Reg.	RandomizedSearch	**0.8399**	0.7642	0.6892	0.6507	0.7386
Logistic Reg.	GridSearch	0.8398	0.7629	0.6880	0.6485	0.7386
Logistic Reg.	Optuna	0.8397	0.7629	0.6880	0.6485	0.7386
Random Forest	RandomizedSearch	0.8327	0.7669	0.6786	0.6557	0.7052
Random Forest	Optuna	0.8301	0.7668	0.6875	0.6480	0.7349
Random Forest	GridSearch	0.8288	0.7591	0.6718	0.6436	0.7052

Table 4: Hyperparameter tuning results — all methods

6.3 Key Observation

Logistic Regression showed negligible improvement after tuning (AUC: 0.8398 → 0.8399), confirming that the default hyperparameters were already near-optimal for this dataset. Random Forest benefited more meaningfully, with Accuracy improving by 1.1% and Precision by 1.8%.

7 API Development — FastAPI

7.1 Architecture

The prediction service was built with FastAPI, a modern Python web framework that generates automatic interactive documentation (Swagger UI) and validates request/response schemas via Pydantic.

Endpoint	Method	Purpose
`/`	GET	Health ping
`/health`	GET	Model load status check
`/predict`	POST	Diabetes risk prediction
`/docs`	GET	Interactive Swagger UI

Table 5: API endpoints

7.2 Request / Response Structure

Input fields: pregnancies, glucose, blood_pressure, skin_thickness, insulin, BMI, diabetes_pedigree_function, age.

Response fields: prediction (0/1), probability (float), label, confidence, BMI (echoed), message, model_time_ms, server_time_ms.

7.3 Middleware

- **CORSMiddleware** — allows cross-origin requests from the browser frontend, with `max_age=86400` to cache preflight responses for 24 hours
- **GZipMiddleware** — compresses responses above 100 bytes to reduce transfer size

7.4 Model Loading Strategy

The model is loaded once at startup via the `lifespan` context manager and held in memory. Subsequent requests do not touch disk or S3 — the model is always ready. S3 is only accessed at container startup.

8 Cloud Deployment

8.1 Infrastructure Overview

Service	Purpose	Region
AWS S3	Model storage (.pkl file)	us-east-1
AWS ECR	Docker image registry	us-east-1
AWS EC2	Application server (t2.micro)	us-east-1
Docker	Application containerisation	—

Table 6: Cloud infrastructure components

8.2 Deployment Steps

1. Trained model uploaded to S3 bucket
2. Application containerised using Docker (python:3.11-slim base)
3. Docker image pushed to AWS ECR
4. EC2 instance (Ubuntu 24.04, t2.micro) launched
5. EC2 pulls image from ECR, loads model from S3 at startup
6. Container runs with `--restart always` for persistence
7. Port 8000 opened in EC2 Security Group

8.3 Docker Strategy

Dependencies were installed in separate `RUN` layers to leverage Docker's layer caching. Only packages required for inference were included — training libraries (CatBoost, SHAP, matplotlib, Streamlit) were excluded, reducing the final image size significantly.

9 Frontend Interface

A five-step web interface (MedPulse AI) was built in plain HTML, CSS, and JavaScript. Key features:

- Dark and light mode toggle
- BMI auto-calculation from weight and height
- Skin thickness auto-estimation from BMI
- Diabetes Pedigree Function calculated from family history questionnaire
- Pregnancy field hidden for male users
- Live monitor tiles updating as the user types blood values
- Animated wave loader during API call
- Result card showing risk score, probability bar, and key metrics
- Latency display with colour-coded quality indicator
- Demo mode fallback if backend is unreachable

10 Latency Analysis

10.1 Measurement Methodology

Two measurement tools were used:

- **browser** — `performance.now()` in JavaScript, measuring total round-trip time including CORS and JSON parsing
- **curl** — direct HTTP measurement, bypassing browser overhead
- **server** — `time.perf_counter()` in Python, measuring server-side processing only

10.2 Measured Results

Component	Time	Share
Model prediction (Logistic Regression)	0.97ms	0.3%
FastAPI overhead (routing/validation)	0.07ms	0.0%
Total server processing	1.04ms	0.3%
TCP handshake (Germany → us-east-1)	139ms	39.9%
Response transfer (us-east-1 → Germany)	111ms	31.9%
Browser overhead (fetch/CORS/JS)	96ms	27.6%
Total browser latency	**348ms**	100%

Table 7: Latency breakdown — Germany to AWS us-east-1

10.3 Key Finding

The server is responsible for just 1.04ms out of 348ms total — 0.3% of the end-to-end latency. The remaining 99.7% is network travel time and browser overhead, neither of which can be optimised in application code.

10.4 Optimisations Applied

1. `max_age=86400` on CORS middleware — caches preflight check for 24 hours, eliminating the duplicate OPTIONS round-trip on repeat requests
2. `GZipMiddleware` — compresses response payload
3. Model loaded once at startup — zero S3 I/O during prediction requests
4. `--restart always` on Docker — container recovers automatically from crashes

10.5 Projected Improvement

Moving EC2 from us-east-1 (Virginia) to eu-central-1 (Frankfurt) would reduce TCP connect time from 139ms to approximately 8ms:

Component	Virginia (now)	Frankfurt (projected)
TCP connect	139ms	8ms
Response transfer	111ms	8ms
Server	1ms	1ms
Browser overhead	96ms	96ms
Total	**348ms**	**~113ms**

Table 8: Latency projection after region migration

11 Tools and Technologies

Category	Tool	Purpose
Language	Python 3.11	Core language
ML Framework	scikit-learn	Models, CV, imputation
Boosting	XGBoost, CatBoost	Ensemble methods
Imbalanced data	imbalanced-learn	SMOTE
Tuning	Optuna	Bayesian hyperparameter optimisation
Tracking	MLflow	Experiment logging and model registry
Explainability	SHAP	Feature importance
API	FastAPI	REST API framework
Validation	Pydantic	Input/output schema validation
Server	Uvicorn	ASGI server
Containerisation	Docker	Application packaging
Image Registry	AWS ECR	Docker image storage
Model Storage	AWS S3	Model artefact storage
Compute	AWS EC2	Application hosting
Frontend	HTML/CSS/JS	User interface

Table 9: Technology stack

12 What Was Learnt

1. **Data leakage is subtle.** Applying SMOTE or imputation before the train/test split is a common mistake that inflates validation scores. Keeping all transformations inside each CV fold is the correct approach.
2. **Simple models can outperform complex ones.** Logistic Regression beat XGBoost and CatBoost on this dataset. Model complexity needs to be justified by data size — on 768 records, a linear model often finds the real signal more reliably.
3. **MLflow makes experiments reproducible.** Without tracking, comparing 10+ experiments across tuning methods is guesswork. MLflow made it trivial to identify the best run and reproduce it exactly.
4. **Docker is about consistency, not just deployment.** The same container runs identically on a laptop and on EC2. Debugging environment differences becomes a non-issue.
5. **Latency is mostly network, not code.** The model runs in under 1ms. The 348ms total latency is almost entirely geography. Profiling with `time.perf_counter()` and `curl` confirmed this clearly.
6. **AWS services fit together as a system.** S3 for storage, ECR for image management, EC2 for compute — each does one thing well. Understanding which service to use for what was a key practical lesson.
7. **CORS is a browser-only concern.** curl has no CORS overhead. The 96ms browser overhead versus direct curl confirmed that CORS preflight was adding a full

extra round-trip on first requests, fixable with `max_age`.

8. **Production readiness is a mindset.** Things like `--restart always`, health check endpoints, structured logging, and proper environment variable handling are not optional extras — they are what separates a working demo from a reliable service.

13 Results Summary

Metric	Value
Best model	Logistic Regression (tuned)
AUC score	0.8399
Accuracy	76.4%
Recall	74.2%
F1 score	0.690
Models compared	5
CV folds	5 (stratified)
Tuning methods used	3 (Grid, Random, Optuna)
Total training experiments	10+
Model inference time	0.97ms
Total server processing	1.04ms
End-to-end browser latency	348ms (Germany → us-east-1)
Projected Frankfurt latency	∼113ms
Deployment	Docker on AWS EC2, model on S3

Table 10: Project results at a glance

14 Conclusion

This project covered the complete lifecycle of a machine learning system — from a raw CSV with missing values and class imbalance, through careful preprocessing, rigorous model comparison, systematic hyperparameter tuning, experiment tracking, REST API development, containerisation, and live cloud deployment.

The most important outcome is not the 84% AUC or the sub-millisecond server response. It is the understanding of how all the pieces fit together: why you cannot apply SMOTE before a CV split, why Docker layer caching matters for deployment speed, why moving a server 6,000 kilometres closer saves 200ms, and why Logistic Regression sometimes beats gradient boosted trees.

These are the kinds of things that are obvious in hindsight and invisible from tutorials alone. They only become clear when you build the whole thing yourself.

Built for learning. All metrics measured on the Pima Indians Diabetes Database.
Not intended for clinical diagnosis or medical advice.